

Exercise 2.1 (Kraft–McMillan inequality) Assume a uniquely decodable code has codeword lengths $l_1, \dots, l_M$. Our goal is to derive Kraft's inequality for uniquely decodable codes:

$$\sum_{j=1}^M 2^{-l_j} \leq 1.$$

a) Prove the following identity (this is easy):

$$\left(\sum_{j=1}^M 2^{-l_j} \right)^n = \sum_{j_1=1}^M \sum_{j_2=1}^M \dots \sum_{j_n=1}^M 2^{-(l_{j_1} + l_{j_2} + \dots + l_{j_n})}.$$

b) Let A_l be the number of concatenations of n codewords that have overall length $l = l_{j_1} + l_{j_2} + \dots + l_{j_n}$ and let $l_{\max} = \max\{l_1, l_2, \dots, l_M\}$ be the maximum length of a codeword. Show that

$$\left(\sum_{j=1}^M 2^{-l_j} \right)^n = \sum_{l=n}^{nl_{\max}} A_l 2^{-l}.$$

c) Using unique decodability, show that $A_i \leq 2^i$ and hence

$$\left(\sum_{j=1}^M 2^{-l_j} \right)^n \leq nl_{\max}.$$

Use this to derive the desired inequality.

Solution:

- a) Perform the multiplication...
- b) The smallest value $l_{j_1} + l_{j_2} + \dots + l_{j_n}$ can take is n , which would happen if all code words had the length 1. The largest value the exponent can take is $nl_{\max}$ where $l_{\max}$ is the maximal codeword length. The summation can then be written as above.
- c) The number of possible binary sequences of length i is 2^i . Since the code is uniquely decodable, we must have $A_i \leq 2^i$ in order to be able to decode. Plugging this into the above bound yields

$$\left(\sum_{j=1}^M 2^{-l_j} \right)^n \leq \sum_{i=n}^{nl_{\max}} 2^i 2^{-i} = nl_{\max} - n + 1 \leq nl_{\max}.$$

We then have

$$\sum_{j=1}^M 2^{-l_j} \leq [nl_{\max}]^{1/n} = \exp \left[\frac{1}{n} \log(nl_{\max}) \right]$$

The exponent goes to zero as $n \rightarrow \infty$ and hence, $\sum_{j=1}^M 2^{-l_j} \leq 1$, Kraft's inequality for uniquely decodable codes.

Exercise 2.2 Let X be an i.i.d. random variable with an infinite alphabet, $\mathcal{X} = \{1, 2, 3, \dots\}$. In addition, let $P(X = i) = 2^{-i}$.

a) What is the entropy of X ?

b) Find an optimal variable length code, and show that it is indeed optimal.

Solution:

a.) By direct calculation,

$$H(X) = \sum_{i=1}^{\infty} -2^{-i} \log(2^{-i}) = \sum_{i=1}^n i 2^{-i} = 2.$$

This is because

$$\sum_{i=1}^{\infty} i x^{i-1} = \frac{1}{(1-x)^2},$$

for all $|x| < 1$. This can be shown by differentiation of the identity $\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$.

b.) Take the code lengths to be $-\log(2^{-1}), -\log(2^{-2}), -\log(2^{-3}), \dots$. Codewords can be

$$\begin{aligned} C(1) &= 0 \\ C(2) &= 10 \\ C(3) &= 110 \\ &\vdots \end{aligned}$$

Exercise 2.3 (Shannon Code) Let X be a random variable distributed on $\{0, 1, 2, \dots, d-1\}$, and let P_X be its pmf. Without loss of generality, we assume $P_X(0) \geq P_X(1) \geq \dots \geq P_X(d-1) > 0$.

A Shannon code for this source is constructed as follows. For each $x \in \mathcal{X}$, we consider the binary representation of the real number $\sum_{x' < x} P_X(x')$, and use the first $\lceil \log_2 \frac{1}{P_X(x)} \rceil$ bits in its fractional part to represent x . E.g., the binary representation of $1/3$ is $'0.01010101\dots'$, and thus we assign it codeword $'01'$.

a) Show that the above code is unique decodable.

b) Produce a Shannon code table for the following source

x	0	1	2	3
$P_X(x)$	$1/2$	$1/6$	$1/6$	$1/6$

and compute the expected code length.

c) Verify that the expected length of Huffman codes for this source is shorter.

Solution:

a.) Denote the codeword representing x as $C(x)$ (namely, for this specific random variable X , Shannon code maps each x to $C(x)$). It suffices to show the above code to be a prefix code. Suppose the opposite, i.e., one of $C(x)$ and $C(\tilde{x})$ is a prefix to the other for some $x < \tilde{x}$. Note that $P_X(x) \geq P_X(\tilde{x})$. Thus, $\lceil \log_2 \frac{1}{P_X(x)} \rceil \leq \lceil \log_2 \frac{1}{P_X(\tilde{x})} \rceil$; namely, $C(x)$ is shorter and must be a prefix to $C(\tilde{x})$. On the other hand, denoting $\ell = \lceil \log_2 \frac{1}{P_X(x)} \rceil$ the length of $C(x)$, we have

$$\left| \sum_{x' < x} P_X(x') - \sum_{x' < \tilde{x}} P_X(x') \right| \geq P_X(x) \geq 2^{-\ell}.$$

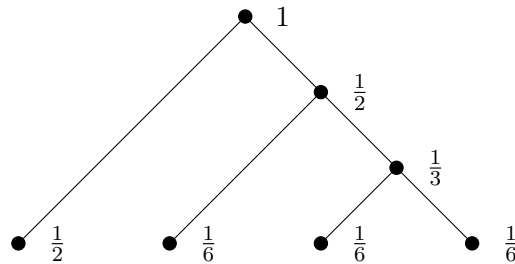
However, this implies a contradiction since the first ℓ bits of these two numbers are same. Hence, the above code must be a prefix code.

b.)

x	$\sum_{x' < x} P_X(x')$	LHS in binary	$\lceil \log_2 \frac{1}{P_X(x)} \rceil$	codeword
0	0	0.00000000	1	0
1	1/2	0.10000000	3	100
2	2/3	0.10101010...	3	101
3	5/6	0.11010101...	3	110

The expected code length is $\frac{1}{2} \cdot 1 + 3 \cdot \frac{1}{6} \cdot 3 = 2$.

c.) The Huffman tree constructed for this case looks like the following:



The expected code length is $\frac{1}{2} \cdot 1 + \frac{1}{6} \cdot 2 + 2 \cdot \frac{1}{6} \cdot 3 = \frac{11}{6} < 2$.

Exercise 2.4 Which of the following sets of codewords can never be valid Huffman codes (for any source probabilities)? Argue why.

- a) {00, 01, 10, 110},
- b) {0, 10, 11},
- c) {00, 01, 10, 11},
- d) {01, 10},
- e) {1, 01, 10}.

Solution:

- a.) Cannot. 110 can be shortened to 11.
- b.) Can.
- c.) Can.
- d.) Cannot. The codewords can be shortened to {0, 1}.
- e.) Cannot. It is not a prefix code.

Exercise 2.5 Consider a random variable X which takes on four possible values with probabilities $(\frac{1}{3}, \frac{1}{3}, \frac{1}{4}, \frac{1}{12})$.

- a) Construct a Huffman code for this source.
- b) Show that there exist two different sets of optimal lengths for the codewords, namely, the codeword length assignments $(1, 2, 3, 3)$ and $(2, 2, 2, 2)$ are both optimal.
- c) Are there optimal codes with codeword lengths for some symbols that exceed the Shannon code length $\lceil \log \frac{1}{p(x)} \rceil$?

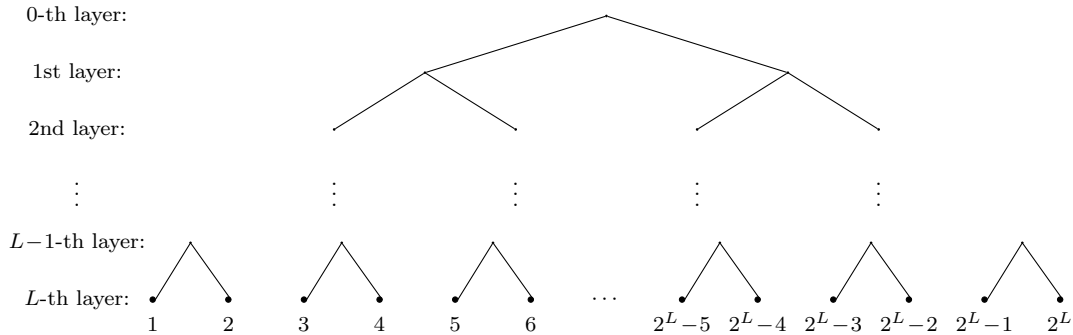
Code 1	Code 2	Symbol	Probability
0	00	1	1/3
11	01	2	1/3
101	10	3	1/4
100	11	4	1/12

Solution:

- Applying the Huffman algorithm gives us the table below, which gives codeword lengths (1, 2, 3, 3) or (2, 2, 2, 2) for the different sets of codewords.
- Both set of lengths (1, 2, 3, 3) and (2, 2, 2, 2) satisfy the Kraft inequality, and they both achieve the same expected length (2 bits) as the Huffman code in step a) for the above distribution. (Notice that Huffman code is optimal.) Therefore they are both optimal.
- The symbol with probability 1/4 has an Huffman code of length 3, which is greater than $\left\lceil \log \frac{1}{p(x)} \right\rceil$. Thus the Huffman code for a particular symbol may be longer than the Shannon code for that symbol. But on average, the Huffman code cannot be longer than the Shannon code.

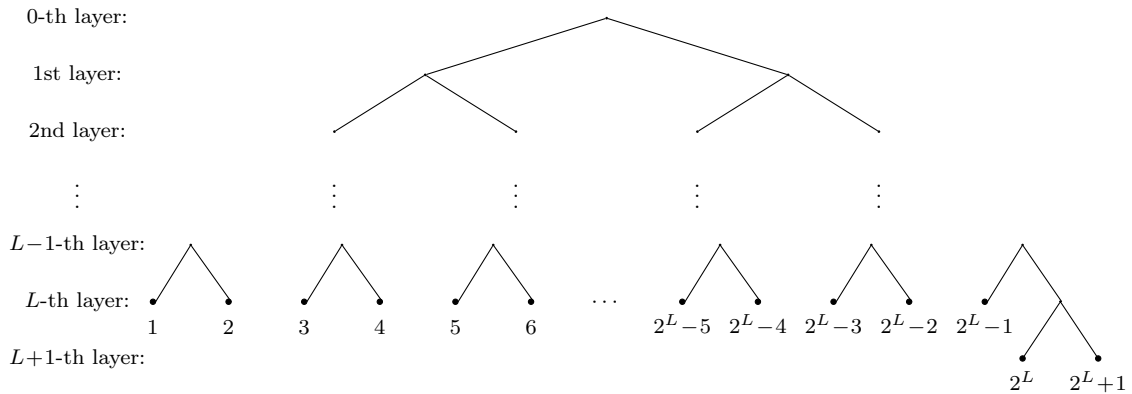
Exercise 2.6 Suppose there is a source X with alphabet $\{1, 2, \dots, n\}$ and each symbol has equal probability. We use the Huffman algorithm to generate a binary code. Calculate the average code length when $n = 2^L$ and $n = 2^L + 1$, where L is a positive integer.

Solution: For $n = 2^L$, the resultant tree of the Huffman algorithm looks like below:



The length of each codeword is L , and thus the average length is L .

For $n = 2^L + 1$, the resultant tree looks like this:



The length of the first $2^L - 1$ (actually arbitrarily picken $2^L - 1$) codewords is L , and the remaining 2 codewords are of length $L + 1$. Thus, the average length is $\frac{2^L - 1}{2^L + 1} \cdot L + \frac{2}{2^L + 1} \cdot (L + 1) = L + \frac{2}{2^L + 1}$.

Exercise 2.7 (Huffman algorithm) Consider a discrete memoryless source X with alphabet $\{1, 2, \dots, M\}$. Suppose that the symbol probabilities are ordered and satisfy $p_1 > p_2 > \dots > p_M$ and also satisfy $p_1 < p_{M-1} + p_M$. Let $l_1, l_2, \dots, l_M$ be the lengths of a prefix-free code of minimum expected length for such a source.

- a) Is the following statement true or false? $l_1 \leq l_2 \leq \dots \leq l_M$. Argue why.
- b) Show that if the Huffman algorithm is used to generate the above code, then $l_M \leq l_1 + 1$.
- c) Show that $l_M \leq l_1 + 1$ for any (not necessarily Huffman generated) prefix-free code of minimum expected length.
Hint: A minimum-expected-length code must be full.
- d) Suppose $M = 2^k$ for some integer k . Show that all codewords must have the same length.
Hint: What does the Kraft inequality look like for an optimal code? Consider the three cases $l_1 = k$, $l_1 < k$ and $l_1 > k$.

Solution:

- a.) For an optimal code, if $p_i > p_j$, then $l_i \leq l_j$. Suppose otherwise, i.e., that $l_i > l_j$, then we can create a new code with lengths $l'_i = l_j$ and $l'_j = l_i$ for symbols i and j respectively. It is then easy to see that the expected cost will decrease. Indeed, the contribution to the expected cost from symbols i and j in the old code is $c_{ij}^{\text{old}} = p_i l_i + p_j l_j$. The contribution to the expected cost from symbols i and j in the new code is $c_{ij}^{\text{new}} = p_i l_j + p_j l_i$. Because $p_i > p_j$ and $l_i > l_j$, we have that $c_{ij}^{\text{old}} > c_{ij}^{\text{new}}$ which is a contradiction and so the initial assumption that $l_i > l_j$ must be false. Hence, $l_i \leq l_j$. This argument can be repeated for all symbols.
- b.) For the Huffman algorithm, we will first merge symbols $M - 1$ and M to form a symbol with probability $p_{M-1} + p_M$. The length of the codeword for this new symbol is $l_M - 1$. Since $p_{M-1} + p_M > p_1$, by part (a), we must have that $l_1 \geq l_M - 1$ which is the desired result.
- c.) A minimum-expected-length code must be full, and thus the codeword for letter M must have a sibling, say letter j . Since $p_j \geq p_{M-1}$, we have $p_j + p_M > p_1$ because of the condition $p_1 < p_{M-1} + p_M$. Let l' be the length of the intermediate node that is parent to j and M . Now $l' \leq l_1$: otherwise, for $l' > l_1$, the codeword for letter 1 could be interchanged with this intermediate node for a reduction in the minimum-expected-length $\bar{L}$. While the length of the codeword for '1' increases from l_1 to l' , the length of the codewords for j and M would reduce from $l' + 1$ to $l_1 + 1$:

$$\begin{aligned} \mathbb{E}[\bar{L}_{\text{old}}] - \mathbb{E}[\bar{L}_{\text{new}}] &= p_1 l_1 + (p_M + p_j)(l' + 1) - p_1 l' - (p_M + p_j)(l_1 + 1) \\ &= (p_M + p_j - p_1)(l' - l_1) > 0, \end{aligned}$$

a contradiction, since $\mathbb{E}[\bar{L}_{\text{old}}]$ was assumed to be minimal. We find $l_M - 1 = l' \leq l_1$.

- d.) An optimal prefix-free code must be full. For full codes, Kraft must be satisfied with equality, i.e., $\sum_{j=1}^{2^k} 2^{-l_j} = 1$. First assume that $l_1 = k$. Then all codewords have length k or $k + 1$, but as mentioned above, the Kraft's inequality can be satisfied with equality (i.e., the code can be full) only if all codewords have length k . If $l_1 > k$, then $l_j > k$ for all j and the Kraft inequality can not be satisfied with equality. Finally, if $l_1 < k$ with $l_j \leq k$, then the Kraft inequality cannot be met. Thus all codewords have length k .

Exercise 2.8 (Compressing English) Consider a source that outputs independent letters according to the frequency that they appear in the English language (use the frequencies listed in Table 1).

- a) Calculate the entropy of this source.
- b) Construct a Huffman code for this source. You may either construct the Huffman code manually according to the algorithm discussed in the lecture, or write a compute program that does this for you.
- c) Compute the expected length of the codeword for this code. How does this compare to the entropy computed in a)?

a	8.4%	b	1.5%	c	2.2%	d	4.2%	e	11.0%	f	2.2%
g	2.0%	h	6.0%	i	7.4%	j	0.1%	k	1.3%	l	4.0%
m	2.4%	n	6.7%	o	7.4%	p	1.9%	q	0.1%	r	7.5%
s	6.2%	t	9.2%	u	2.7%	v	0.9%	w	2.5%	x	0.1%
y	2.0%	z	0.1%								

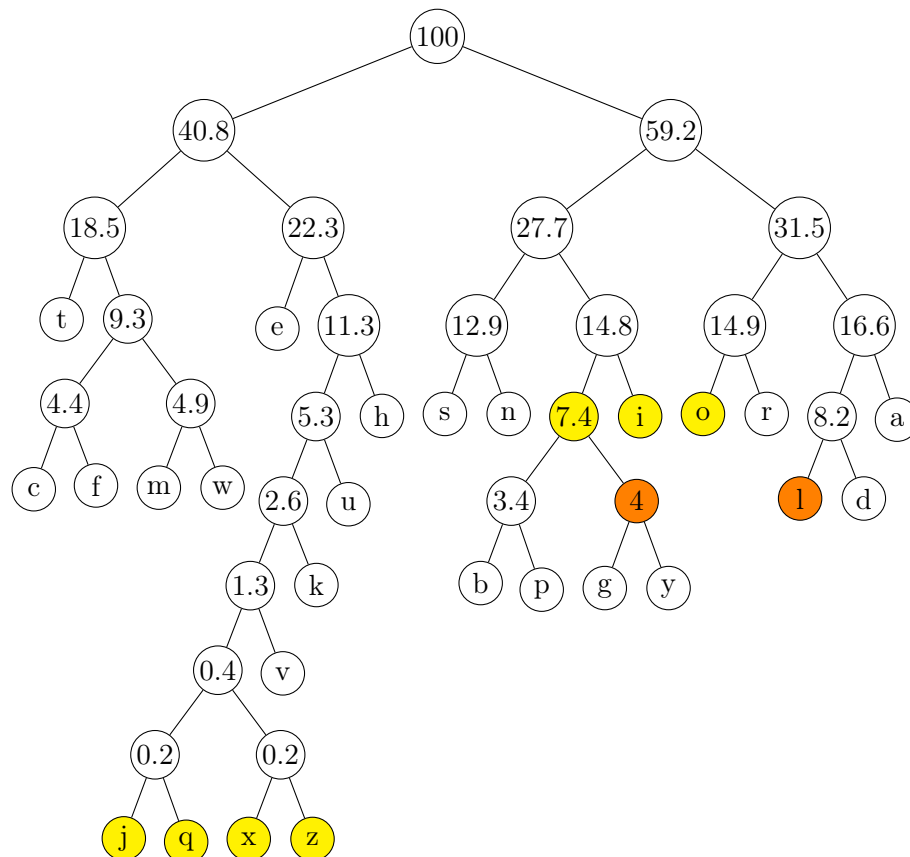
Table 1: Statistical distribution of letters in the English language. Source: https://en.wikipedia.org/wiki/Letter_frequency, but normalized so that they add up to 100%.

Solution:

- a.) Let $\mathcal{X}$ denote the alphabet and $X \in \mathcal{X}$. Then the entropy of this source is given by

$$H(X) = \sum_{x \in \mathcal{X}} p(x) \log \frac{1}{p(x)} \approx 4.19.$$

- b.) A Huffman tree without edge labels is shown below. For readability, the leaves are only labeled with the corresponding letter. Note that certain nodes have the same weight such that the respective subtrees are interchangeable: $4 \leftrightarrow l$, $7.4 \leftrightarrow i \leftrightarrow o$, and $j \leftrightarrow q \leftrightarrow x \leftrightarrow z$. To derive a set of codewords, each pair of edges (b_1, b_2) linking a node with its children needs to be assigned values 0 and 1 according to $(b_1, b_2) \rightarrow (0, 1)$ or $(b_1, b_2) \rightarrow (1, 0)$.



One possible Huffman code: (See the Matlab code “huffman.m”)

a	0111	b	001000	c	10100	d	01101	e	110	f	10101
g	001010	h	1111	i	0011	j	111000010	k	111001	l	01100
m	10110	n	0001	o	0100	p	001001	q	111000011	r	0101
s	0000	t	100	u	11101	v	1110001	w	10111	x	111000000
y	001011	z	111000001								

Listing 1: huffman.m

```

1 function [code,compression,avelen]=huffman5(p)
2 %HUFFMAN5
3 %HUFFMAN CODER FOR V5
4 % Format [CODE,COMPRESSION,AVELEN]=HUFFMAN5(P)
5 %
6 % P is the probability (or number of occurrences) of each alphabet symbol
7 % CODE gives the huffman code in a string format of ones and zeros
8 % COMPRESSION gives the compression rate
9 % AVELEN gives the expected length of the code
10 %
11 % Huffman5 works by first building up a binary tree (eg p =[ .5 .2 .15 .15])
12 %
13 %      a_1      a_4
14 %      1/      1/
15 %      /      /
16 %    b3      b1
17 %     \    /  \
18 %    0\ 1/  0\
19 %      b2      a_3
20 %       \
21 %      0\
22 %        a_2
23 %
24 % Such that the tree always terminates at an alphabet symbol and the
25 % symbols furthest away from the root have the lowest probability.
26 % The branches at each level are labeled 0 and 1.
27 % For this example CODE would be
28 %      1
29 %      00
30 %      010
31 %      011
32 % and the compression rate 1.1111
33 % Sean Danaher University of Northumbria at Newcastle UK 98/6/4
34
35 p=p(:)/sum(p);    %normalises probabilities
36 c=huff5(p);
37 code=char(getcodes(c,length(p)));
38 compression=ceil(log(length(p))/log(2))/(sum(code' ~= ' ')*p);
39 avelen=sum(sum(code' ~= ' ').*p);
40 %
41 function c=huff5(p)
42 % HUFF5 Creates Huffman Tree
43 % Simulates a tree structure using a nested cell structure
44 % P is a vector with the probability (number of occurrences)

```

```

45 % of each alphabet symbol
46 % C is the Huffman tree. Note Matlab 5 version
47 % Sean Danaher 98/6/4 University of Northumbria, Newcastle UK
48
49 c=cell(length(p),1); % Generate cell structure
50 for i=1:length(p) % fill cell structure with
    1,2,3...n
51 c{i}=i; % (n=number of
    symbols in alphabet)
52 end
53 while size(c)-2 % Repeat till only two
    branches
54 [p,i]=sort(p); % Sort to ascending
    probabilities
55 c=c(i); % Reorder
    tree.
56 c{2}={c{1},c{2}};c(1)=[]; % join branch 1 to 2 and prune 1
57 p(2)=p(1)+p(2);p(1)=[]; % Merge Probabilities
58 end
59 %-----
60 function y= getcodes(a,n)
61 % Y=GETCODES(A,N)
62 % Pulls out Huffman Codes for V5
63 % a is the nested cell structure created by huffcode5
64 % n is the number of symbols
65 % Sean Danaher 98/6/4 University of Northumbria, Newcastle UK
66 global y
67 y=cell(n,1);
68 getcodes2(a,[])
69 %-----
70 function getcodes2(a,dum)
71 % GETCODES(A,DUM)
72 %getcodes2
73 % called by getcodes
74 % uses Recursion to pull out codes
75 % Sean Danaher 98/6/4 University of Northumbria, Newcastle UK
76
77 global y
78 if isa(a,'cell')
79 getcodes2(a{1},[dum 0]);
80 getcodes2(a{2},[dum 1]);
81 else
82 y{a}=setstr(48+dum);
83 end

```

c.) The expected length which is 4.22 is larger than the entropy which is 4.19.

Exercise 2.9 (Code with Prefix) We usually like codes to be prefix-free as otherwise we do not know when a codeword ends and decoding might no longer be unique. A simple alternative way to overcome this problem is to add a special symbol that indicates the end of a codeword. In this exercise we thus use codewords comprised of “0” and “1” that always end with “_”.

- a) Construct a code for this source in the following way: the two most frequent letters according to Table 1 are assigned codewords of length $1 + 1$, the next four most frequent letters codewords of length $2 + 1$, etc.
- b) Compute the expected length of the codewords.
- c) The code you arrived at is very similar to a famous code that has been in use since the 1830s. Which code is that? Compare the expected length of codewords of your code to that of this historical code.
- Hint:** Replace “0” by “.” and “1” by “–” in your codewords.

Solution:

- a.) (See the Matlab code “ex3_2.m”)

a	b	c	d	e	f	g	h	i	j	k
00_	0101_	0000_	011_	0_	0001_	0010_	010_	10_	1000_	0110_
l	m	n	o	p	q	r	s	t	u	v
100_	111_	000_	11_	0100_	1001_	01_	001_	1_	101_	0111_
w	x	y	z							
110_	1010_	0011_	1011_							

Listing 2: ex3_2.m

```

1  %%%Exercise 3.2 prefix code%%
2
3  pp=[8.4,1.5,2.2,4.2,11,2.2,2.0,6,7.4,0.1,1.3,4,2.4,6.7,7.4,1.9,0.1,7.5, ...
4      6.2,9.2,2.7,0.9,2.5,0.1,2,0.1];
5  A='a':'z';
6  AA=mat2cell(A,1,ones(26,1));
7  AAA=[];
8
9  %sort probabilities and record the index
10 [pp0,pindex]=sort(pp,'descend');
11
12 %generate codewords
13 code={};
14 for i=1:4
15     code(2^i-1:(2^i-1)*2)=cellstr(dec2bin(0:2^i-1));
16 end
17 code=code(1:26);
18
19 %combine symbols and codewords
20 for i=1:26
21     AA(2,pindex(i))=cellstr([cell2mat(code(i)), '_']);
22 end
23
24 %expected length
25 Sum=0;
26 for i=1:26
27     Sum=Sum+pp(i)*length(cell2mat(AA(2,i)));
28 end
29 Sum=Sum/100;

```

```

30
31 %expected time
32 time=0;
33 for i=1:26
34     t0=2*length(find(cell2mat(AA(2,i))=='0'));
35     t1=4*length(find(cell2mat(AA(2,i))=='1'));
36     time=time+pp(i)/100*(t0+t1);
37 end
38 time=time+3

```

b.) Expected length: 3.433.

c.) Morse code. The expected length of Morse code is 3.558, which is longer than that of our code.

Exercise 2.10 Consider a DMS with a two symbol alphabet $\{a, b\}$ where $p_X(a) = 2/3$ and $p_X(b) = 1/3$. Let $X^n = (X_1, \dots, X_n)$ be a string of symbols emitted by the source with $n = 100,000$. Let $W(X_j)$ be the surprisal for the j -th source output, i.e., $W(X_j) = -\log \frac{2}{3}$ for $X_j = a$ and $-\log \frac{1}{3}$ for $X_j = b$. Define $W(X^n) = \sum_{j=1}^n W(X_j)$.

- Find the variance of $W(X_j)$. For $\epsilon = 0.01$, evaluate a bound on the probability of the typical set $\mathcal{A}_\epsilon^{(n)}$ using Chebyshev's inequality.
- Let N_a be the number of a 's in the string $X^n = (X_1, \dots, X_n)$. The random variable N_a is the sum of n i.i.d. random variables. What are these?
- Express the random variable $W(X^n)$ as a function of N_a . Note how this depends on n .
- Express the typical set in terms of bounds on N_a . Use Chebyshev's inequality to derive bounds on the probability of the typical set, using properties of N_a instead of $W(X_j)$.
Hint: You may write $\mathcal{A}_\epsilon^{(n)} = \{x^n : \alpha < N_a < \beta\}$ and calculate α and β .
- Find $P[N_a = i]$ for $i = 0, 1, 2$. Find the probability of each individual string x^n for those values of i . Find the particular string x^n that has maximum probability over all sample values of X^n . What are the next most probable n -strings? Give a brief discussion of why the most probable n -strings are not regarded as typical strings.

Solution:

- a.) For notational convenience, we will denote the log pmf random variable by W . Now, note that W takes on values $-\log 2/3$ with probability $2/3$ and $-\log 1/3$ with probability $1/3$. Hence,

$$\text{Var}(W) = \mathbb{E}[W^2] - \mathbb{E}[W]^2 = \frac{2}{9}.$$

The bound on the typical set, as derived using Chebyshev's inequality is

$$P(X^n \in \mathcal{A}_\epsilon^{(n)}) \geq 1 - \frac{\sigma_W^2}{n\epsilon^2}.$$

Substituting the values of $n = 10^5$ and $\epsilon = 0.01$, we obtain

$$P(X^n \in \mathcal{A}_\epsilon^{(n)}) \geq 1 - \frac{1}{45} = \frac{44}{45}$$

Loosely speaking this means that if we were to look at sequences of length 100,000 generated from our DMS, more than 97% of the time the sequence will be typical.

- b.) The random variable N_a is the sum of n i.i.d. random variables Y_i , $N_a = \sum_{i=1}^n Y_i$ where Y_i 's are Bernoulli with $P(Y_i = 1) = 2/3$.
- c.) The probability of a particular sequence X^n where a appears N_a times is $(2/3)^{N_a}(1/3)^{n-N_a}$. Hence,

$$W(X^n) = -\log p_{X^n}(x^n) = -\log[(2/3)^{N_a}(1/3)^{n-N_a}] = n \log 3 - N_a.$$

- d.) For a sequence X^n to be typical, it must satisfy

$$\left| -\frac{1}{n} \log p_{X^n}(x^n) - H(X) \right| < \epsilon$$

From (a) the source entropy is $H(X) = \mathbb{E}[W(X)] = \log 3 - 2/3$, and substituting in ϵ and $W(X^n)$ from part (d), we get

$$\left| \frac{N_a}{n} - \frac{2}{3} \right| \leq 0.01$$

Note the intuitive appeal of this condition! It says that for a sequence to be typical, the proportion of a 's in that sequence will be very close to the probability that the DMS generates an a . Plugging in the value of n in the above equation, we get the bounds on

$$65,667 \leq N_a \leq 67,666.$$

- e.)

$$P(N_a = 0) = \left(\frac{1}{3}\right)^n, \quad P(x^n = b^n) = \left(\frac{1}{3}\right)^n.$$

$$P(N_a = 1) = n \left(\frac{2}{3}\right) \left(\frac{1}{3}\right)^{n-1}, \quad \text{the probability of each } x^n \text{ is } \left(\frac{2}{3}\right) \left(\frac{1}{3}\right)^{n-1}.$$

$$P(N_a = 2) = \frac{n(n-1)}{2} \left(\frac{2}{3}\right)^2 \left(\frac{1}{3}\right)^{n-2}, \quad \text{the probability of each } x^n \text{ is } \left(\frac{2}{3}\right)^2 \left(\frac{1}{3}\right)^{n-2}.$$

The particular string x^n with maximum probability is a^n :

$$P(x^n = a^n) = \left(\frac{2}{3}\right)^n.$$

The next most probable n -strings are x^n with 1 symbol b and $(n-1)$ symbol a 's.

The typical strings should have around $\frac{2}{3}n$ symbol a 's and around $\frac{1}{3}n$ symbol b 's. The most probable n -strings are usually far from this situation.

Exercise 2.11 Consider a DMS $\mathbf{X}$ which produces sequences of letters from an alphabet $\mathcal{X}$ such that the entropy of each letter is $H(X)$. Assume also that for any $k \in \mathbb{N}$, we have a Huffman code for k -tuples produced by this source. That is, a Huffman code that maps sequences $x^k \in \mathcal{X}^{\times k}$ of length k to codewords $C_k(x^k) \in \{0,1\}^*$ with lengths $\ell_k(x^k)$.

- a) Give an upper bound on $\mathbb{E}[\ell_k(X^k)]$ for X^k produced from this source in terms of the entropy $H(X)$. You might want to consider the case $k = 1$ first and recall properties of the Shannon code and its relation to the Huffman code.
- b) Use the weak law of large numbers to show that, for any $k \in \mathbb{N}$,

$$\lim_{m \rightarrow \infty} P \left[\frac{1}{mk} \sum_{i=1}^m \ell_k(X_i^k) \geq H(X) + \frac{2}{k} \right] = 0.$$

- c.) Now consider a rate $R = H(X) + \delta$ for any $\delta > 0$. Choose $k = \lceil \frac{2}{\delta} \rceil + 1$. For each $m \in \mathbb{N}$, design a block encoder for blocks of length $n = km$, that is, an encoder $e_n : X^n \rightarrow [2^{\lfloor Rn \rfloor}]$ and a decoder d_n creating an estimate $\hat{X}^n$ such that

$$\lim_{n \rightarrow \infty} P[X^n \neq \hat{X}^n] = 0.$$

You have now shown that optimal variable-length codes are also optimal for block-coding.

Solution:

- a.) Huffman codes are prefix codes, and hence

$$\mathbb{E}[\ell_k(X^k)] = \bar{\ell}_k(X^k) \leq H(X^k) + 1 = kH(X) + 1.$$

- b.) Consider the random variable $\frac{1}{k}l_k(X^k)$. Since we know that

$$kH(X) \leq \bar{\ell}_k \leq kH(X) + 1,$$

we have that the mean μ of $\frac{1}{k}l_k(X^k)$ is bounded as

$$H(X) \leq \mu \leq H(X) + \frac{1}{k}.$$

Then, by the weak law of large numbers, we have

$$\begin{aligned} \lim_{m \rightarrow \infty} P\left[\left|\frac{1}{mk} \sum_{i=1}^m \ell_k(X_i^k) - \mu\right| \geq \frac{1}{k}\right] &= 0, \\ \lim_{m \rightarrow \infty} P\left[\frac{1}{mk} \sum_{i=1}^m \ell_k(X_i^k) \geq H(X) + \frac{2}{k}\right] &= 0. \end{aligned}$$

- c.) Consider a Huffman code for blocks of size $k = \lceil \frac{2}{\delta} \rceil + 1$. Now consider m such blocks for any $m \in \mathbb{N}$. Let $n = mk$. By the previous part, we have that

$$\lim_{m \rightarrow \infty} P\left[\sum_{i=1}^m \ell_k(X_i^k) \geq mkH(X) + 2m\right] = 0.$$

That is, the length of the encoded message L_n for m blocks of size $k = \lceil \frac{2}{\delta} \rceil + 1$ satisfies

$$\begin{aligned} \lim_{n \rightarrow \infty} P\left[L_n \leq nH(X) + \frac{2n}{k}\right] &= 1 \\ \implies \lim_{n \rightarrow \infty} P\left[L_n \leq nH(X) + \frac{2n\delta}{2 + \delta}\right] &= 1. \end{aligned}$$

For a rate $R = H(X) + \delta$, we have

$$\begin{aligned} n\left(H(X) + \frac{2\delta}{2 + \delta}\right) &= n\left(R - \delta + \frac{2\delta}{2 + \delta}\right) \\ &= n\left(R - \frac{\delta^2}{2 + \delta}\right) \\ &\leq \lfloor nR \rfloor. \end{aligned}$$

Hence a rate of $H(X) + \delta$ is achievable for any $\delta > 0$.

A possible encoding/decoding process of Huffman encoding is described in Algorithm 2.1 of the notes for $x^k \in \mathcal{X}^k$ with probability $p_{x^k} = P_{X^k}(x^k)$.

Exercise 2.12 (Coding with side information) Consider a memoryless source $(\mathbf{X}, \mathbf{Y})$ that produces in each iteration two random variables, X_i and Y_i , where X_i is private information and Y_i is public information. The pairs (X_i, Y_i) are i.i.d. following a pmf P_{XY} .

We are looking for a fixed-length block code that compresses the private information $X^n = (X_1, X_2, \dots, X_n)$ using the public information $Y^n = (Y_1, Y_2, \dots, Y_n)$ such that the code can be decoded asymptotically error-free with help of the public information.

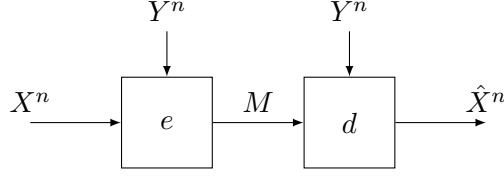


Figure 1: An $(n, 2^L)$ -code for a source with side information is given by an encoder, $e : (X^n, Y^n) \rightarrow M$, and decoder, $d : (M, Y^n) \rightarrow \hat{X}^n$. The codeword $M \in \{0, 1\}^L$ is of length L .

We define $R^*(\mathbf{X}|\mathbf{Y})$ as the infimum over all rates R such that there exists a sequence of $(n, 2^{nR})$ -codes satisfying

$$\lim_{n \rightarrow \infty} P[X^n \neq \hat{X}^n] = 0, \quad \text{where} \quad \hat{X}^n = d_n(e_n(X^n, Y^n), Y^n).$$

- a) Determine $R^*(\mathbf{X}|\mathbf{Y})$, by intuitive or formal arguments, for the simple cases where
 - i.) X and Y are independent,
 - ii.) $X = Y$;
- b) By explicitly constructing a code for the source (X, Y) using codes for the sources Y and X (with side information Y), show that $R^*(\mathbf{X}, \mathbf{Y}) \leq R^*(\mathbf{X}|\mathbf{Y}) + R^*(\mathbf{Y})$.
- c) Show the converse, $R^*(\mathbf{X}|\mathbf{Y}) \geq H(X|Y)$, using Fano's inequality.

Hint: You may use the following sequence of inequalities, which needs to be verified.

$$H(X^n|\hat{X}^n) \geq H(X^n|Y^n M) \tag{1}$$

$$= H(X^n M|Y^n) - H(M|Y^n) \tag{2}$$

$$\geq H(X^n M|Y^n) - L \tag{3}$$

$$\geq H(X^n|Y^n) - L. \tag{4}$$

- d) Give a formal proof or a sketch of a proof that $R^*(\mathbf{X}|\mathbf{Y}) \leq H(X|Y)$. For this purpose, consider the conditional typical set $\mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y})$, given as

$$\left\{ (x^n, y^n) \in \mathcal{X}^n \times \mathcal{Y}^n : \left| \frac{1}{n} \log \frac{1}{P_{X^n|Y^n}(x^n|y^n)} - H(X|Y) \right| \leq \epsilon \right\}.$$

This establishes that $R^*(\mathbf{X}|\mathbf{Y}) = H(X|Y)$.

Solution:

- a.) i.) $R^*(\mathbf{X}|\mathbf{Y}) = H(X)$. If X and Y are independent, it means Y does not provide useful side information about X .
- ii.) $R^*(\mathbf{X}|\mathbf{Y}) = 0$. $X = Y$ means that the decoder can exactly recover X^n simply from the side information Y^n .

- b.) A code for the source (X, Y) can be constructed by concatenating the codes for sources Y and X (with side information Y). The error probability is given by

$$\begin{aligned} P\left[(\hat{X}^n, \hat{Y}^n) \neq (X^n, Y^n)\right] &= P\left[\hat{Y}^n \neq Y^n \vee \hat{X}^n \neq X^n\right] \\ &\leq P\left[\hat{Y}^n \neq Y^n\right] + P\left[\hat{X}^n \neq X^n\right] \rightarrow 0, \text{ as } n \rightarrow \infty. \end{aligned}$$

Let L_Y and $L_{X|Y}$ denote the number of bits of the optimal (shortest) codes for the sources Y and X with side information Y , respectively. Then we have

$$R^*(\mathbf{X}, \mathbf{Y}) \leq \frac{L_{X|Y} + L_Y}{n} = R^*(\mathbf{X}|\mathbf{Y}) + R^*(\mathbf{Y}).$$

The inequality follows from the fact that we could encode XY by encoding Y and $X|Y$ separately, such that the optimal L_{XY} satisfies $L_{XY} \leq L_{X|Y} + L_Y$ (when encoded separately, the decoder could first decode Y , and then use Y to decode $X|Y$ as in the setting depicted in Figure 1).

- c.) We first verify the sequence of inequalities. Eq. (1) is the data-processing inequality applied to the fact that $\hat{X}^n$ is computed from Y^n and M . Eq. (2) is the chain rule for conditional entropy. Eq. (3) follows from the dimension bound for $|M| \leq 2^L$. Finally, Eq. (4) uses the chain rule and the fact that $H(M|X^n Y^n) \geq 0$.

Consider now any sequence of $(n, 2^L)$ -codes that satisfy $\epsilon_n = P[\hat{X}^n \neq X^n] \rightarrow 0$ in the limit $n \rightarrow \infty$. By Fano's inequality and using the given sequence of inequalities, we have

$$H(\epsilon_n) + \epsilon_n n \log |\mathcal{X}| \geq H(X^n | \hat{X}^n) \geq H(X^n | Y^n) - L.$$

Hence, as $n \rightarrow \infty$,

$$\begin{aligned} \frac{L}{n} &\geq \frac{1}{n}(H(X^n | Y^n) - H(\epsilon_n) - \epsilon_n n \log |\mathcal{X}|) \\ &\geq \frac{1}{n}H(X^n | Y^n) - \frac{1}{n} - \epsilon_n \log |\mathcal{X}| \\ &= H(X|Y) - \frac{1}{n} - \epsilon_n \log |\mathcal{X}| \\ &\rightarrow H(X|Y). \end{aligned}$$

Since this holds for any sequence of codes, we conclude that $R^*(\mathbf{X}|\mathbf{Y}) \geq H(X|Y)$.

- d.) Use the definition of $\mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y})$ in the hint, and further define

$$\mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y})\big|_{y^n} := \left\{x^n \in \mathcal{X}^n \mid (x^n, y^n) \in \mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y})\right\}.$$

By definition, for any $y^n \in \mathcal{Y}^n$, we can establish that

$$\begin{aligned} P_{X^n|Y^n}(x^n|y^n) &\geq 2^{-n(H(X|Y)+\epsilon)} \\ \implies 1 &\geq \sum_{x^n: (x^n, y^n) \in \mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y})} P_{X^n|Y^n}(x^n|y^n) \geq \left|\mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y})\big|_{y^n}\right| \cdot 2^{-n(H(X|Y)+\epsilon)} \\ \implies \left|\mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y})\big|_{y^n}\right| &\leq 2^{n(H(X|Y)+\epsilon)}. \end{aligned}$$

Furthermore, let $Z_i = \log \frac{1}{P_{X|Y}(X_i|Y_i)} - H(X|Y)$ and we have

$$\begin{aligned}
P \left[(X^n, Y^n) \in \mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y}) \right] &= P \left[\left| \frac{1}{n} \log \frac{1}{P_{X^n|Y^n}(X^n|Y^n)} - H(X|Y) \right| \leq \epsilon \right] \\
&= P \left[\left| \frac{1}{n} \log \frac{1}{\prod_{i=1}^n P_{X|Y}(X_i|Y_i)} - H(X|Y) \right| \leq \epsilon \right] \\
&= P \left[\left| \frac{1}{n} \sum_{i=1}^n \log \frac{1}{P_{X|Y}(X_i|Y_i)} - H(X|Y) \right| \leq \epsilon \right] \\
&= 1 - P \left[\left| \frac{1}{n} \sum_{i=1}^n \log \frac{1}{P_{X|Y}(X_i|Y_i)} - H(X|Y) \right| > \epsilon \right] \\
&= 1 - P \left[\left| \frac{1}{n} \sum_{i=1}^n Z_i \right| > \epsilon \right].
\end{aligned}$$

Since Z_i are i.i.d. and have zero mean, by the weak law of large numbers, we have

$$\lim_{n \rightarrow \infty} P \left[(X^n, Y^n) \in \mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y}) \right] = 1 - \lim_{n \rightarrow \infty} P \left[\left| \frac{1}{n} \sum_{i=1}^n Z_i \right| > \epsilon \right] = 1.$$

Encoder e :

$$e(x^n|y^n) = \begin{cases} m(x^n|y^n) & (x^n, y^n) \in \mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y}), \\ 0^L & (x^n, y^n) \notin \mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y}). \end{cases}$$

where $m(x^n|y^n)$ is the index of x^n in the set $\mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y})|_{y^n}$, and thus the size of the messages alphabet is upper bounded by $2^{n(H(X|Y)+\epsilon)}$.

Decoder d : Given y^n, m , output any x^n such that $m = m(x^n|y^n)$.

Then the error probability is given by

$$P \left[\hat{X}^n \neq X^n \right] = 1 - P \left[(X^n, Y^n) \in \mathcal{A}_\epsilon^{(n)}(\mathbf{X}|\mathbf{Y}) \right] \rightarrow 0 \text{ for } n \rightarrow \infty.$$

This implies that $R \leq \frac{1}{n} \cdot \log 2^{n(H(X|Y)+\epsilon)} = H(X|Y) + \epsilon$ is achievable. Since $\epsilon > 0$ can be chosen to be arbitrarily small, we have

$$R^*(\mathbf{X}|\mathbf{Y}) \leq H(X|Y).$$

Exercise 2.13 (Slepian–Wolf Coding) Let X and Y be a pair of jointly distributed random variables on finite alphabets $\mathcal{X}$ and $\mathcal{Y}$, respectively.

The rate pair (R_1, R_2) is said to be achievable for DMS (X, Y) if there exists a sequence of $(n, 2^{nL_1}, 2^{nL_2})$ -codes with encoders $e_{1,n}$, $e_{2,n}$ and decoder d_n such that

$$\lim_{n \rightarrow \infty} P\{(\hat{X}^n, \hat{Y}^n) \neq (X^n, Y^n)\} = 0$$

where

$$(\hat{X}^n, \hat{Y}^n) = d_n(M_1, M_2), \quad M_1 = e_{1,n}(X^n), \text{ and } M_2 = e_{2,n}(Y^n)$$

are the reconstructed source and codewords respectively.

Prove that, for any (R_1, R_2) achievable, it must hold that

$$\begin{aligned}
R_1 &\geq H(X|Y), \\
R_2 &\geq H(Y|X), \quad \text{and} \\
R_1 + R_2 &\geq H(X, Y).
\end{aligned}$$

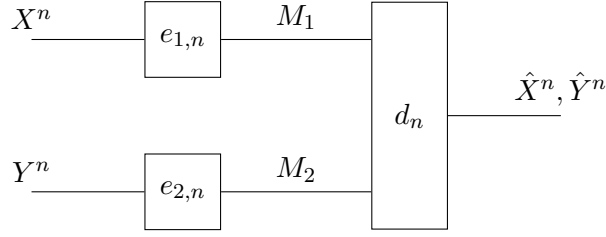


Figure 2: An $(n, 2^{nL_1}, 2^{nL_2})$ -separately-encoded jointly-decoded source code consists of a pair of encoders $e_1 : \mathcal{X}^n \rightarrow \{0, 1\}^{nL_1}$ and $e_2 : \mathcal{Y}^n \rightarrow \{0, 1\}^{nL_2}$, and a decoder $d : \{0, 1\}^{nL_1} \times \{0, 1\}^{nL_2} \rightarrow \mathcal{X}^n \times \mathcal{Y}^n$.

Solution: The separately-encoded jointly decoded source code on $\mathcal{X}$ and $\mathcal{Y}$ is a special case of the jointly-encoded jointly decoded source code on $\mathcal{X} \times \mathcal{Y}$. By considering (M_1, M_2) as a single message, the last equation follows directly from the proof in the lecture notes. We are only going to show the proof for the first equation below. The proof for the second equation follows symmetrically.

Suppose (R_1, R_2) is achievable. For any $\epsilon > 0$, there exists some $N \in \mathbb{N}$ such that for all $n > N$, we have

$$P\{\hat{X}^n \neq X^n\} \leq P\{(\hat{X}^n, \hat{Y}^n) \neq (X^n, Y^n)\} < \epsilon.$$

By Fano's inequality, we have

$$I(\hat{X}^n : X^n) = nH(X) - H(\hat{X}^n | X^n) \geq nH(X) - \epsilon n \log |\mathcal{X}| - 1.$$

On the other hand, by how M_1 and M_2 are constructed, we have

$$\begin{aligned} I(\hat{X}^n : X^n) &\stackrel{(a)}{\leq} I(M_1, M_2 : X^n) \\ &\stackrel{(b)}{\leq} I(M_1 : X^n) + I(M_2 : X^n) \\ &\stackrel{(c)}{\leq} I(M_1 : X^n) + I(X^n : Y^n) \\ &\leq nR_1 + nI(X : Y) \end{aligned}$$

where (a) follows from the Markov chain $X^n - (M_1, M_2) - \hat{X}^n$, (b) follows from the Markov chain $M_2 - X^n - M_1$ and (c) follows from the Markov chain $X^n - Y^n - M_2$. Thus,

$$nH(X) - \epsilon n \log |\mathcal{X}| - 1 \leq nR_1 + nI(X : Y),$$

which implies

$$R_1 \geq H(X|Y) - \epsilon \cdot \log |\mathcal{X}| - \frac{1}{n}.$$

Since above holds from all $\epsilon > 0$, and all $n > N_\epsilon$, one must have $R_1 \geq H(X|Y)$.